

DEMO: Original synthetic editorial-evaluation material. All numerical results and protocol outcomes are authored fixtures, not real experiments or a publication.

Publishing Base-Relative Tile Patches as Complete Raster Revisions

Abstract

A raster update may arrive tile by tile while the display requires a complete revision. Publishing each replacement immediately can expose a mixture that belongs to neither the old nor the new revision. Full-snapshot staging already prevents this failure. We assemble changed-tile patches privately, sharing unchanged tiles from a named, pinned base and publishing only after every declared replacement validates. In synthetic sparse-update fixtures on a constrained link, this method preserves the full-snapshot coherence criterion while reducing median completion from 87 to 17 ms and wire volume from 1026 to 68 KiB. Dense updates and sparse updates on a fast local link instead complete more slowly. Reconnect controls explain why base validation and acknowledgement after publication matter. The contribution is a specific transfer design and its conditional cost comparison, not a new atomic-publication primitive or demonstrated service performance.

1. Arrival is not a complete revision

Suppose a dashboard displays tiles A and B from revision 10. Revision 11 replaces both. If A arrives first and its pointer becomes visible immediately, the display can combine new A with old B. Revision tags on individual messages identify their destination but do not stop this mixed view. Withholding the replacements until both are ready preserves revision 10 during transfer and exposes revision 11 afterwards.

The existing service already supports that publication boundary. It stores immutable tile versions behind a root of pointers, publishes roots atomically, and lets the renderer capture one root at each frame's start. Copying roots, reference counting, dirty-tile tracking and checksums are inherited facilities. A staged full snapshot also provides coherent views; coherence alone therefore cannot establish an advantage for a patch protocol.

The remaining question is whether unchanged tiles can be reused without making incremental network arrival visible, and when doing so reduces completion cost. We compare full snapshots, immediate tile installation and staged patches. The method explains the dependence on a particular client base; the evaluation keeps coherence, transfer volume and overhead together. Raster values are assumed computed already. Their scientific accuracy is outside this study.

2. Assemble privately against the published base

Each raster has 64 tiles of 4096 four-byte cells: $64 \times 4096 \times 4 = 1,048,576$ payload bytes. Full snapshot (FS) stages the entire raster, validates its packets and switches the visible root. Immediate tiles (IT) transfer changed tiles tagged with a target revision, installing each pointer upon arrival. Staged patches (SP) declare a base revision, target revision and changed-tile set in a manifest.

SP pins the visible base and constructs a private pending root. Unchanged pointers are shared from that base; changed coordinates receive validated target replacements. In the two-tile example, the visible pair remains (a0,b0) while a1 waits in private state. After b1 arrives and both validate, the next reader can see (a1,b1). An unchanged tile C remains shared. Target coherence means matching the producer's target root, not assigning every tile the target's creation timestamp: a tile created in revision 8 may legitimately remain in revision 11.

Before publication, the commit checks the pinned base. Out-of-order packets may enter the named target's private state, but late valid packets cannot update a newer visible root. Matching duplicates are idempotent; a conflicting duplicate or failed size, checksum or tile-ID check rejects the patch. Incomplete or invalid patches are discarded,

and disconnect discards uncommitted state. In each case the last coherent root remains visible. Display reads do not wait for pending packets, although clients need not display the same revision at the same wall-clock instant.

The published root also determines which unchanged tiles a later patch may reuse. The client therefore acknowledges only after publication. Acknowledging the first packet could let the server derive a later patch from a root the client has not published. The server retains manifests for at most eight completed revisions; a missing, wrong or unverifiable acknowledged base triggers FS, including first connection without a base. When combining several retained steps, the server unions their touched coordinates and sends each coordinate once, using its target-root value. An intermediate value would assemble the wrong target. A tile that changes and returns to its old bytes may remain in the union, so coalescing is not minimal-difference compression.

Private assembly has a storage cost. The pointer table is 512 bytes and pending descriptors/reference counters add 4096 bytes. Old tiles and replacements coexist until readers release old roots; dense SP can require a full frame of replacements. FS similarly retains the old frame and a new full frame. No bound on an old reader's lifetime is established.

3. Evaluation: compare qualifying views before delays

All table entries are synthetic fixtures. Within each workload/link pair, methods receive the same 120 target roots, lossless tile values and packet schedule. Completion runs from producer target publication until all required packets are applied or staged, not from first-byte arrival. Coherence counts runs without a non-target mixed root at any prescribed reader interleaving from request to completion, rather than only checking the eventual root. Those interleavings are finite.

The three constrained-link patterns change 4, 24 and 60 tiles; only the four-tile pattern also appears on the fast local link. Queue budget and retry deadline remain 32 packets and 20 ms throughout. An older 64-packet exploratory run is excluded. Serialization and transport implementations are identical except for payload selection and manifest handling, and compression is disabled. There is one fixed schedule per case, with no repetition, confidence interval, stochastic loss model or CPU microarchitecture study.

FS and SP pass every listed coherence check, so their paired costs answer the performance question. IT remains in Table 1 to show the consequence of exposing arrivals immediately.

Table 1. Complete synthetic records. Completion is median/p95 ms; wire volume is median KiB including framing; preparation is median ms. Retained payload is peak MiB, excluding pointer tables and descriptors, and assumes one reader finishes within one display cycle. KiB and MiB are binary units. IT timing does not qualify as coherence-preserving performance.

Changed tiles / link	Method	Coherent / total	Median ms	p95 ms	Wire KiB	Retained MiB	Prepare ms
4 / constrained	FS	120/120	87	106	1026	2.00	0.18
4 / constrained	IT	71/120	13	26	66	1.06	0.05
4 / constrained	SP	120/120	17	31	68	1.06	0.32
24 / constrained	FS	120/120	89	110	1026	2.00	0.18
24 / constrained	IT	40/120	47	69	386	1.38	0.07
24 / constrained	SP	120/120	51	74	388	1.38	0.37
60 / constrained	FS	120/120	91	111	1026	2.00	0.18
60 / constrained	IT	31/120	83	104	962	1.94	0.09
60 / constrained	SP	120/120	97	120	964	1.94	0.44

Changed tiles / link	Method	Coherent / total	Median ms	p95 ms	Wire KiB	Retained MiB	Prepare ms
4 / fast local	FS	120/120	2.2	3.1	1026	2.00	0.18
4 / fast local	IT	71/120	1.3	2.2	66	1.06	0.05
4 / fast local	SP	120/120	2.6	3.9	68	1.06	0.32

On the constrained link with four changed tiles, SP reduces median completion from 87 to 17 ms, an 80.5% reduction relative to FS, and wire volume from 1026 to 68 KiB, a 93.4% reduction. For the moving 24-tile pattern, median completion falls from 89 to 51 ms and wire volume from 1026 to 388 KiB. The p95 values improve in both cases as well.

The comparison reverses for 60 changed tiles: SP takes 97 versus 91 ms, despite sending 964 versus 1026 KiB. The fast-local sparse case likewise takes 2.6 versus 2.2 ms despite its large byte reduction; both cases also have worse p95 delay. Preparation is higher for SP in every pair, reaching 0.44 ms against FS's 0.18 ms. Extra assembly work is a plausible explanation for limited returns, but these whole-method comparisons do not isolate its causal contribution or assign every delay difference to bytes.

IT completes sooner but produces only 71, 40 and 31 coherent runs out of 120 in the constrained cases, and 71/120 on the fast local link. Its eventual packet completion cannot compensate for an intermediate view that is not a producer revision.

A separate suite contains 28 hand-authored protocol traces, all matching SP's target-root oracle. It covers first connection, empty patches, delayed final tiles, reordering, matching and conflicting duplicates, corruption, disconnect, late reconnect packets, base eviction and two pinned readers. Eight eviction/reconnect traces use FS fallback. There is no recovery-latency summary or basis for extending sparse-path byte savings to fallback. Removing base checks fails 2/6 reconnect traces; acknowledging on the first packet fails 1/6 after coalescing. These are functional controls, not timing ablations.

4. Discussion

Full snapshots already meet the display requirement. SP earns a place beside them when reusing unchanged tiles saves enough transfer work to repay dependent assembly. The sparse and moving-front constrained fixtures show that opportunity; the dense and fast-local fixtures show that fewer bytes alone cannot select the faster method. The four cases do not establish a crossover threshold or an adaptive selection policy.

Dependence on a client base also limits where savings apply. History eviction and reconnect can require a complete snapshot, while slow reader release can prolong retention after publication. The functional suite checks the former path without quantifying recovery cost. The payload figures assume prompt release and establish no memory bound for stalled readers. These are distinct limits on adopting the same patch mechanism.

The state is volatile display state, not a durable transaction or crash-recovery database. Allocation-failure injection and process-crash tests are absent; authentication, client-server security and Byzantine behaviour are outside scope. No literature survey establishes novelty, and no deployment or statistical-significance claim follows from authored fixtures. The synthetic comparisons identify questions for a real implementation study, not measured answers about a production service.

5. Conclusion

A base-relative patch can reuse unchanged tiles while keeping incomplete arrivals private until a complete target root is ready. The supplied fixtures distinguish that mechanism from immediate installation and compare it fairly with already-coherent full snapshots. Its value depends on how much changes and how costly transfer is; dense updates, fast links, fallback and reader retention limit the advantage.